

Guía para el uso de git y Github

GitHub es un servicio gratuito de internet que actúa como repositorio online de código. Github facilita la colaboración en proyectos donde intervienen muchas personas y resulta útil para controlar las distintas versiones del código y facilitar el trabajo simultáneo de varios desarrolladores.

Para poder hacer uso de Github, son necesarios dos pasos, que se describirán a continuación:

1. Instalación de Git

Git es un programa que funcionará localmente. Este se encarga de gestionar todos los archivos dentro del repositorio local y efectuar todas las operaciones de interés para subir los archivos al repositorio remoto, mezclar los cambios y avisa de todos los posibles conflictos.

Su instalación es relativamente sencilla, simplemente hay que descargar el programa de su [repositorio oficial](#) y seguir las instrucciones para la instalación. Durante este proceso, el asistente de la instalación propondrá muchas opciones para la configuración. Si no se dispone de conocimiento sobre lo que cambiar las recomendaciones por defecto, es mejor no tocar nada y dejar las opciones por defecto.

Una vez que tenemos el programa instalado, podremos hacer una prueba muy sencilla sobre su funcionamiento. Al escribir en la terminal:

```
git --version
```

Podremos ver como respuesta algo del estilo:

```
git version 2.34.1
```

Esto es un indicativo de que Git se ha instalado adecuadamente y que funciona correctamente, así que podremos proceder con el resto de pasos en estas instrucciones.

2. Uso de GitHub

Como ya se ha comentado antes, GitHub es un servicio de internet. Para poder acceder a este, es necesario disponer de una cuenta. A esta cuenta estarán asociados todos los repositorios de los que se será propietario y se podrá utilizar (con su dirección de correo asociada o el nombre de usuario) para que otros usuarios puedan compartir sus repositorios privados.

Además, se puede colaborar con GitHub en los repositorios públicos, los cuales son accesibles por todo el mundo (pero el propietario siempre tendrá la potestad de decidir qué pasa con el código del repositorio).

Cuando ya se dispone de un repositorio remoto donde se busca trabajar, bien porque este se haya creado y esté vacío, o bien comenzando a colaborar con un grupo de personas, es necesario seguir unos cuantos pasos para tener la combinación git-GitHub funcionando.

2.1. Vinculación del usuario de GitHub a git

Los primeros pasos que hay que seguir son indicar a git qué usuario y credenciales deberá utilizar para acceder a los repositorios remotos. Esto lo podremos hacer introduciendo en la terminal estas dos líneas:

```
git config --global user.name "tu-usuario"
git config --global user.email "tu@email.com"
```

Ahora, para acceder a un repositorio remoto que ya se ha creado, simplemente hay que seguir los siguientes pasos:

1. En el caso de que el repositorio sea privado, solicitar permiso al dueño.
2. Ejecutar, dentro de la carpeta donde se quiere copiar el repositorio remoto, la siguiente línea:

```
git clone https://github.com/usuario/nombre-del-repo
```

Donde el link es el link del repositorio, por lo que **usuario** en este caso es el usuario dueño del repositorio. Si es la primera vez, puede que git pida iniciar sesión.

3. Verificar que el repositorio se ha copiado correctamente dentro de los archivos locales.

A partir de ahora, cada vez que se busque copiar todos los cambios del repositorio remoto al local, el comando que habrá que ejecutar es:

```
git pull
```

Que se encargará de copiar todos los cambios que se han hecho en el repositorio remoto y efectuarlos en el repositorio local. Esta es la forma de actualizar el repositorio local con el remoto y ponerse al día.

Con esto, toda la configuración básica de GitHub está hecha, y se puede proceder a describir con más detalle su uso.

3. Flujo de trabajo con Git

Para poder aprovechar las ventajas de git, es importante comprender su funcionamiento. Git funciona utilizando el concepto de **ramas** o *branches*, que permiten controlar el flujo de los cambios en el proyecto. Pero antes de esto, hay que establecer dos conceptos fundamentales: el del repositorio local y el repositorio remoto.

El repositorio local no es más que la carpeta del proyecto que se encuentra en el dispositivo desde el que se está trabajando, y existe uno por cada miembro que colabore en el proyecto. Esto funciona exactamente de la misma manera que cualquier conjunto de archivos y directorios en una máquina local. En principio, se conservan todas las libertades para poder crear, mover y modificar archivos. Veremos más adelante por qué es recomendable tener algún tipo de cuidado al hacer esto, pero en principio no hay ninguna limitación.

Por otro lado, está el repositorio remoto. Este es un lugar único en la nube donde todo el código puesto en común del proyecto vive. Sin embargo, esto no quiere decir que en el repositorio remoto solo pueda existir una versión del proyecto.

A medida que se vaya progresando con el desarrollo del proyecto en el repositorio local, se pueden ir guardando los cambios en el historial de versiones. Esto se puede conseguir muy fácilmente con dos líneas en la terminal:

```
git add .  
git commit -m "indicaciones claras y concisas de los cambios"
```

Con esto hemos hecho dos cosas. En primer lugar, `git add .` habrá tomado nota de todos los cambios y los habrá pasado al “staging area”, mientras que `git commit` guardará todos los cambios del “staging area”. Con esto, tendremos hecho un **commit** en nuestro repositorio local. Para pasar estos cambios al repositorio remoto, podemos utilizar:

```
git push
```

Que se encargará de subir todos los **commits** nuevos al repositorio remoto.

3.1. Staging Area

3.1.1. Funcionamiento General

El Staging Area es un “buffer” dentro del cual se guardan los archivos indicados con el comando

```
git add "archivo"
```

Pasar a este comando el punto, `git add .` sirve para indicar que se deben añadir al Staging Area todos los archivos que hayan sido modificados del directorio. Cuando se ejecuta

```
git commit -m "mensaje"
```

Se genera una nueva versión en el repositorio local, con todo lo que había entonces en el Staging Area.

3.1.2. Quitar archivos del Staging Area

Para eliminar archivos del Staging Area antes de hacer un commit, se puede utilizar el comando:

```
git rm --cached nombre/archivo
```

GitHub tiene un límite de 100 MB para el tamaño de los archivos. Si hay un archivo que exceda este tamaño en el repositorio local, el comando `git push` fallará, con el mensaje correspondiente de que el archivo mencionado excede el límite de tamaño. Por esto, puede ser interesante quitar estos archivos del Staging Area.

3.2. Log de cambios

Para acceder a todos los cambios que se han hecho en el directorio, se puede acceder al log que git hace con cada commit:

```
git log --oneline
```

En este caso, se ha añadido la etiqueta **-oneline** para que aparezca por línea la ID del commit y su mensaje. Con esto, se pueden copiar las ID's de cada commit para, por ejemplo poder resetear el Staging Area o los cambios a un estado anterior.

Sin embargo, si ha llegado a saltar el mensaje de error al hacer el **git push**, esto quiere decir que ya existe un commit con los cambios que incluyen el archivo grande. Los commits guardan cambios respecto al commit anterior, y ya se ha guardado localmente un commit con este archivo, lo único que se puede hacer es resetear el historial de commits hasta justo antes de este commit erróneo.

3.3. Eliminar commits erróneos

En el caso de haber hecho un commit erróneo (y tener añadido un archivo demasiado grande, por ejemplo), hay varias formas de volver atrás. Para ello, se utiliza el comando:

```
git reset
```

Este comando puede funcionar directamente. Sin embargo, es preferible especificar también la forma en la que se quiere resetear, y a qué estado se quiere resetear. El comando completo quedaría:

```
git reset --type commit-ID
```

Donde **type** puede ser **hard**, **mixed** o **soft**, y **commit-ID** es la ID del commit al que se quiere resetear. A continuación se explica la diferencia (a nivel práctico) entre los distintos modos de **git reset**.

3.3.1. **-hard**

Esta opción restaurará el estado del repositorio local a exactamente el estado del commit indicado. Se perderán los cambios en los archivos, los archivos nuevos creados desde entonces, y los cambios añadidos al Staging Area.

3.3.2. **-soft**

Esta opción mantendrá los archivos sin cambiar, pero tomará como referencia para los cambios el commit al que se retrocede, por lo que en el Staging Area estarán todos los cambios que se han hecho desde entonces.

3.3.3. **-mixed**

Esta opción es igual que la **-soft**, es decir, que no se modificarán los archivos dentro del repositorio local, pero sí que se sobrescribe el Staging Area con la del commit al que se

retrocede, por lo que si se quieren guardar los cambios que se han hecho hasta ahora, habrá que volver a escribir `git add .`

3.3.4. Ejemplo solución de commit erróneo

Supongamos que se está trabajando con Patran-Nastran y se ha generado un archivo `.f06` muy grande que supera el límite de 100 MB. Una vez alcanzado un hito del trabajo, se ejecutan los comandos:

```
git add .
git commit -m "Trabajo terminado"
git push
```

Y el comando que falla es `git push`, con el aviso de que hay un archivo que excede el tamaño máximo permitido por GitHub. Si ahora se elimina el archivo `.f06`, y se vuelve a hacer `git add.`, `git commit` y `git push`, seguirá habiendo errores, porque el commit anterior en este caso sigue conteniendo el archivo grande.

La forma de solucionar esto, evitando perder todo el progreso conseguido, es utilizando `git reset --mixed`. El flujo de trabajo sería:

```
git reset --mixed origin/main
git add .
git rm --cached *.f06
git commit -m "descripcion de los cambios"
git push
```

Lo que se ha hecho ha sido:

1. Resetear para volver a un commit donde no existe el archivo `.f06` (o existe una versión anterior no tan grande)
2. Añadir al Staging Area todo
3. Eliminar del Staging Area los archivos `.f06`
4. Commit estándar
5. Push estándar

3.4. Archivo `.gitignore`

Cuando se está trabajando con programas que generan archivos muy grandes (como Patran-Nastran o Ansys), o se generan carpetas de *builds* que son específicas para cada máquina y no deben compartirse en el repositorio, la forma más cómoda de evitar que estos archivos entren al Staging Area y a los commits es diciéndolo a git que ignore los cambios en estos archivos o carpetas, y que no los añada nunca al repositorio remoto.

La forma de hacer esto es con un archivo `.gitignore`, que se encarga de especificar qué archivos ignorar. Además, soporta sintaxis para seleccionar archivos por extensión o carpetas/directorios por nombre.

Para incluir esto, simplemente hay que crear un documento de texto en el editor de preferencia con nombre y extensión simplemente `.gitignore` (el punto es importante). A continuación se presenta un ejemplo de lo que podría ser interesante añadir para estudiantes de grado y máster en la ETSIAE:

```
# Python
__pycache__/
*.pyc
*.pyo
*.pyd
.env
.venv/

# Lsp Cache
.cache/

# Builds
build/
dist/
*.exe

# OS junk
.DS_Store
Thumbs.db

# Nastran
*.f06
```

3.5. Branches

Esto nos deja con una posible fuente de conflictos en el caso de un desarrollo colaborativo. Si dos personas hacen modificaciones en el mismo archivo en sus respectivos repositorios locales y quisieran subir ambos sus cambios al repositorio remoto, habrá conflictos. Estos surgirán en git indicando que hay varios cambios nuevos en el mismo archivo, y que no sabe cuáles son los que debe conservar (y entonces comenzará el tedioso trabajo de ir línea a línea eligiendo cuál conservar). Para evitar esto, se introduce el concepto de las ramas o *branches*, como se puede ver representado en la Figura 1

A la hora de añadir cualquier tipo de funcionalidad nueva al proyecto, la mejor práctica es hacer una nueva rama desde el estado anterior (normalmente la rama principal, o **main branch**) y hacer allí todos los commits que sea necesario. Cuando la funcionalidad haya sido debidamente implementada (recomendable que se haya hecho añadiendo archivos nuevos y evitando modificar los originales en la mayor medida de lo posible), se podrá combinar esta rama con la principal.

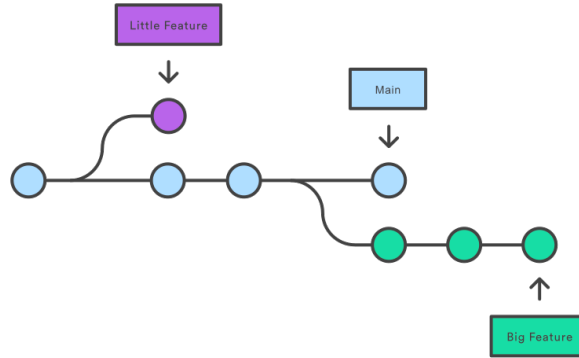


Fig 1: Diagrama del funcionamiento de las ramas en GitHub

3.5.1. Cómo abrir una rama nueva

Cuando se estima que se van a desarrollar una serie de cambios nuevos, puede resultar interesante abrir una rama nueva para poder separar el desarrollo de estas características del de las demás. En estos casos, lo más interesante es declarar una nueva rama en el repositorio remoto y comenzar a subir los cambios allí. Los comandos para hacer esto son:

```
git checkout -b nombre-rama
```

Con esto, habremos creado una nueva rama y estaremos trabajando en ella. Esto se podrá confirmar con `git branch`, que proporcionará una lista de las ramas disponibles en el repositorio.

A medida que se progresa con el desarrollo, se continuará con el flujo de trabajo típico de:

```
git add .
git commit -m "descripcion de los cambios"
```

Hasta que se llegue a un hito importante, y entonces la forma de sincronizar los cambios al repositorio remoto es con:

```
git push -u origin nombre-rama
```

Aquí, la etiqueta `-u` quiere decir: “a partir de ahora, todas los *commits* que haga deben ir a la rama **nombre-rama**”. Con esto, habremos subido los cambios al repositorio remoto en la rama nueva, y los cambios que subamos serán en esta misma rama nueva. No obstante, para proyectos no muy grandes, es interesante trabajar en la rama principal. Los mismos git o GitHub se encargarán de conservar el historial de versiones y facilitarán volver atrás en caso de “liarla” y que surja la necesidad de revertir los cambios.

4. Configuración de git desde el escritorio remoto UPM

Uno de los usos más útiles de esta herramienta, es para poder trabajar cómodamente entre varias estaciones de trabajo. Por ejemplo, FLUENT es un programa que está disponible en el escritorio UPM, y su instalación es compleja. Por otro lado, trabajar en fluent genera una

grandísima cantidad de archivos y su transferencia mediante una memoria portátil (USB) o mandando el directorio de trabajo no es rápido. Para solucionar esto, la recomendación es activar git en el escritorio remoto, que viene ya instalado.

4.1. Configuración previa en GitHub

Para tener acceso al repositorio de GitHub desde una máquina cualquiera, es necesario iniciar sesión. Es muy recomendable, en lugar de iniciar sesión de la forma estándar, utilizar un *token* personalizado para reducir los permisos concedidos a los estrictamente necesarios. De esta forma, se reduce el riesgo de compartir por error el *token* de acceso.

Para conseguir el *token*, el procedimiento a seguir desde la página web de GitHub es el siguiente:

1. En GitHub, haciendo click sobre el icono del perfil a la derecha, ir a ajustes → Ajustes de desarrollador (Developer settings)
2. Personal access tokens → Fine-grained tokens → Generate new token
3. Daremos un nombre y una descripción al token, y seleccionaremos **All repositories**.
4. En Permissions, daremos permisos de lectura y escritura (en cada permiso) para:
 - Pull requests
 - Commit statuses
 - Contents

Se añadirá el permiso de metadata en read-only, es normal.

5. Generaremos el *token* y en la pantalla a continuación, es muy importante copiarlo y guardarlo en un sitio seguro, ya que esta será la contraseña que siempre se debe usar para iniciar sesión en una máquina remota.

Con el *token* ya generado y disponible, podemos iniciar git en el escritorio virtual. La recomendación más cómoda para hacer esto, es abrir una terminal en el escritorio. Para ello, se puede hacer **click derecho** sobre un espacio vacío en el escritorio → abrir en terminal. En esta línea de comandos introduciremos los típicos:

```
git init
git config --global user.name "nombre-usuario"
git config --global user.email "tucorreo@gmail.com"

git clone https://github.com/nombre-usuario/nombre-repositorio
```

En este momento, nos pedirá git que iniciemos sesión para clonar el repositorio. Se seleccionará la opción de iniciar sesión mediante token, y a continuación se puede introducir el token (recomiendo copiarlo y pegarlo)

```
cd nombre-repositorio
```

Con esta retahíla de comandos habremos:

1. Iniciado git en el escritorio,

2. Configurado el usuario y la cuenta,
3. Hecho una copia en el escritorio del repositorio remoto, e iniciado sesión con el token de acceso personal, y
4. Hemos cambiado el directorio de trabajo a dentro del repositorio (equivalente a abrir la consola dentro del repositorio).

Con esto, estaremos trabajando en la rama principal del repositorio remoto, y a partir de ahora, podremos guardar los cambios con el flujo de trabajo típico de:

```
git add .  
git commit -m "descripcion de los cambios"  
git push
```

5. Colaboradores en GitHub

Cuando varias personas van a trabajar de forma colaborativa en un repositorio, GitHub proporciona dos formas distintas de facilitar este tipo de trabajo. Ahora se explicará lo sencillo que es añadir colaboradores en un repositorio.

5.1. Requisitos previos

Es necesario que:

- El repositorio exista (no necesariamente que contenga nada).
- Todos los colaboradores tengan una cuenta de GitHub.

5.2. Añadir a colaboradores

El propietario del repositorio deberá:

- Ir a los ajustes del repositorio → colaboradores → añadir gente
- Escribir la dirección de correo electrónico asociada con la cuenta de un colaborador o su nombre de usuario.
- Hacer click sobre añadir ‘nombre-usuario’.

El colaborador deberá entonces aceptar la solicitud de colaboración. Una vez hecho esto, el propietario podrá ajustar el nivel de permisos de cada uno de los colaboradores. Para permitir la contribución, deberá seleccionarse **read/write**.